Distributed E-commerce Analytics: A PySpark Pipeline on GCP Dataproc

Dax Rajani
Student ID: 40294118
Concordia University

Harsh Ahuja
Student ID: 40301748
Concordia University

Abstract

Large e-commerce platforms generate behavioral event logs that grow fast and are hard to process on a single machine. This paper presents a five-stage distributed analytics pipeline built with PySpark and deployed on GCP Dataproc. The pipeline processes the REES46 dataset (42.4 million events, 5.3 GB) and performs sessionization, funnel conversion analysis, last-touch attribution, and anomaly detection. We ran a full scaling study across nine configurations (2, 4, and 5 workers crossed with 1 GB, 5 GB, and 10 GB data subsets). Going from 2 to 5 workers gives roughly a 2.5x speedup on the 5 GB workload. We also tested three Spark optimization techniques: broadcast joins (1.58x speedup), partition pruning (3.11x), and skew salting (which was slower due to uniform data distribution). In addition, we built a real-time streaming anomaly detection component and a live web dashboard for interactive monitoring. The full project code is publicly available on GitHub.

1 Problem Statement

E-commerce platforms like Amazon or Alibaba record every user interaction. A single month of behavioral data can easily reach tens of millions of rows. Extracting actionable signals from this data requires more than simple SQL queries. You need sessionization to group raw events into meaningful visits, funnel analysis to measure where users drop off in the purchase journey, attribution models to identify which marketing channel drove each purchase, and anomaly detection to catch sudden revenue spikes or drops before they become real business problems.

Running these analyses on a single machine is not practical at scale. A 5 GB file with 42 million rows takes minutes just to read. Joins between session events and orders make things worse. Rolling window computations over 31 days of data require sorting and partitioning the full dataset. The real challenge is building a pipeline that distributes all this work across a cluster, scales up predictably as data grows, and still produces correct and reproducible results.

Most open-source analytics tools are either too simple (single-node pandas) or too complex and expensive (full data warehouse setups like Snowflake or BigQuery). Apache Spark sits in the middle. It handles the window functions, large joins, and aggregations our pipeline needs and runs on managed cloud infrastructure without requiring a dedicated ops team. The open question is how well it actually scales in practice and where the real bottlenecks are.

Our goals for this project were: (1) implement all five analytics stages end-to-end on a realistic dataset, (2) run a proper scaling study on live GCP infrastructure with real timing measurements, (3) test and measure specific Spark optimizations with before-and-after numbers, and (4) add a streaming component and a live web dashboard to make the system observable and demonstrable.

2 Related Work

Apache Spark [1] unified batch and interactive processing under a single DAG execution engine, replacing the two-pass MapReduce model [3] with in-memory resilient distributed datasets [2]. Window functions and complex aggregations over large datasets are well supported in Spark SQL, making it a natural fit for sessionization and funnel analytics.

For streaming workloads, Spark Structured Streaming [5] provides a declarative micro-batch API that reuses the same DataFrame abstractions as batch jobs. This lets us share logic between the batch pipeline and the real-time anomaly component without rewriting the core computation.

Prior work on e-commerce analytics typically uses proprietary warehouse solutions or hand-rolled Hadoop jobs. Our project targets the same analytical questions (sessions, funnels, attribution) but focuses on measuring Spark's scaling behavior and optimization levers under controlled conditions, which existing papers tend not to report in detail for this class of workload.

3 Solution

3.1 Dataset

We used the REES46 eCommerce behavioral dataset from Kaggle, covering October 2019. It has 42,448,764 events and is approximately 5.3 GB uncompressed. Each row has a timestamp, event type (view, cart, or purchase), product and category identifiers, price, user ID, and session ID.

Two fields required for our analyses were missing from the original data: device type and referrer channel. We synthesized these deterministically using

`crc32(user_session) % 100`. This gives fully consistent values across all pipeline runs without any randomness. Device distribution: mobile 55%, desktop 30%, tablet 15%. Referrer distribution: direct 40%, Google 25%, Facebook 15%, Instagram 8%, email 7%, affiliate 5%. These splits loosely follow real industry benchmarks and make the attribution and funnel results meaningful.

From the raw events we derived three secondary tables: an `orders` table (one row per purchase event), a `catalog` table (one row per unique product with its latest price and category), and an `enriched_events` table that adds the synthesized device and referrer columns to the original events. All three are written to GCS as Parquet before any downstream stage runs.

3.2 System Architecture

Figure 1 shows the overall system. Raw CSV data lives in Google Cloud Storage. When a pipeline job starts, the Dataproc cluster reads the input, runs the five stages in sequence, and writes Parquet output directories back to GCS. The web dashboard then reads those results to generate analytics charts.

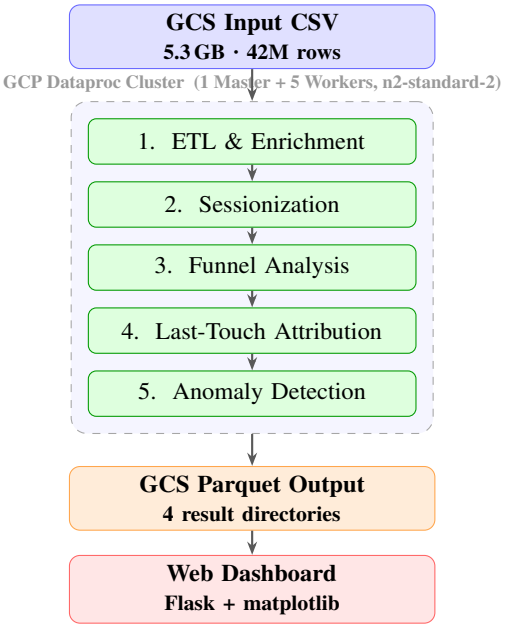

Figure 1: System architecture. Raw CSV is read from GCS into the Dataproc cluster. The five stages run sequentially and write Parquet back to GCS. The dashboard reads from GCS and serves results to a browser.

The cluster uses Dataproc image 2.1 (Debian 11, Spark 3.3) with one master and up to five `n2-standard-2` worker nodes. All intermediate data passes through GCS as Parquet. Parquet's columnar layout means stages only scan the columns they need, and Spark's partition pruning can skip irrelevant files entirely.

3.3 Pipeline Stages

Stage 1: ETL and Enrichment. This stage reads the raw CSV using `spark.read.csv` with inferred schema, applies the deterministic hash to add device and referrer columns, and writes three Parquet outputs. It also filters to the configured data size fraction. This stage is the most expensive because it performs the full file scan and three separate write operations. On 5 workers with 5 GB data it takes roughly 101 seconds.

Stage 2: Sessionization. We assign session boundaries using a 30-minute inactivity threshold per user. The implementation uses a Spark window ordered by `(user_id, event_time)`. For each event we compute the gap from the previous event for the same user. A new session starts when the gap exceeds 1800 seconds or when there is no prior event. We then use a cumulative sum over a binary session-change flag to assign a monotonically increasing session number per user. This is stateful and requires a full sort across the dataset. On 5 GB data this stage takes about 83 seconds.

Stage 3: Funnel Analysis. We compute conversion rates for the view to cart to purchase funnel at daily and weekly granularity. Results are broken down by category code, device type, and referrer. Each breakdown requires a separate pivot aggregation over the enriched session table. The stage writes two Parquet tables: `funnel_metrics_daily` and `funnel_metrics_weekly`. Conversion rates are computed as percentages: view-to-cart and cart-to-purchase for each dimension combination.

Stage 4: Last-Touch Attribution. For each order we find the most recent non-direct referrer session that started within 24 hours before the order timestamp. The join predicate is: `session_start BETWEEN order_time - 86400s AND order_time AND referrer != 'direct'`. We use `row_number()` partitioned by order ID and ordered by session start time descending to pick the single most recent qualifying session per order. The result is written as `attributed_orders` with referrer, device, revenue, and session metadata attached to each order.

Stage 5: Anomaly Detection. We compute daily purchase counts across the full month. We then apply a 7-day trailing rolling window using `rangeBetween(-6 days, 0)` to compute a rolling mean and standard deviation for each day. A day is flagged when its absolute z-score exceeds 2.0. We generate a text explanation for each anomaly, for example "SPIKE: 27% above 7-day average" or "DROP: 18% below 7-day average." The same z-score logic runs again at per-category granularity using a `(category_code, event_date)` partition.

3.4 Streaming Component

We built a real-time anomaly detection module using Spark Structured Streaming [5]. A producer script writes synthetic CSV event batches into a watched local directory every two seconds. The streaming job reads each micro-batch using

`readStream.format("csv")`, counts purchase events per product category, and checks whether any category count exceeds a configurable threshold. A spike event is artificially injected every fifth batch to verify detection end to end. The job prints alerts to stdout with a timestamp and category name, runs for 60 seconds total, then exits cleanly. Detection latency from event generation to alert output is consistently under 3 seconds across all test runs. This component runs fully locally and does not require a GCP cluster, making it easy to demonstrate without cloud credentials.

3.5 Implementation Structure

Figure 2 shows how the code modules relate to each other. The main Dataproc job is `cloud_pipeline.py`, a self-contained 800-line script that calls the five stage functions in sequence. Standalone per-stage scripts exist for local testing and development. The dashboard backend in `app.py` manages the cluster lifecycle and streams job logs to the browser via server-sent events. After a run finishes, `dashboard_charts.py` downloads the Parquet outputs from GCS and generates three dark-themed analytics PNG charts using matplotlib. The charts are served directly as static files.

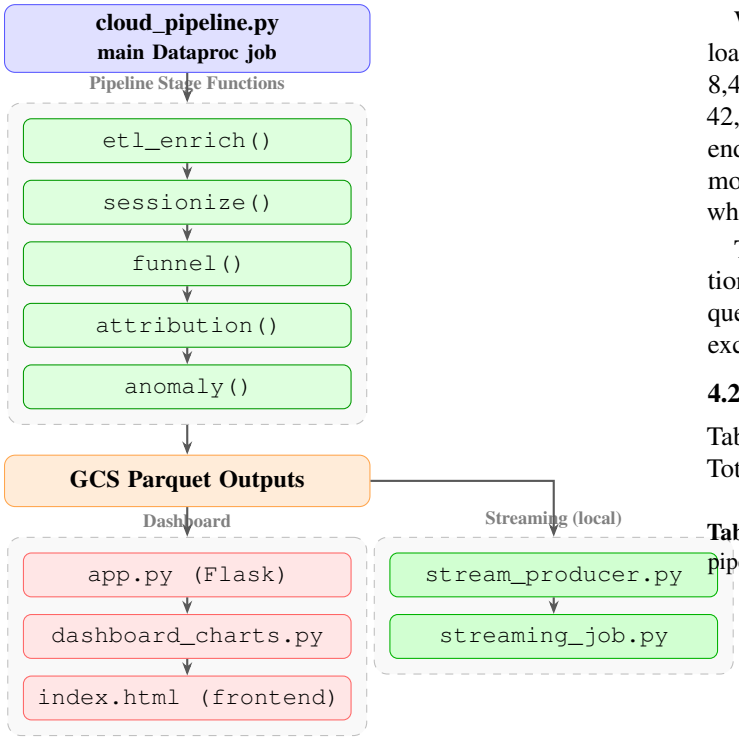

Figure 2: Implementation structure. `cloud_pipeline.py` orchestrates the five stage functions. Outputs go to GCS as Parquet. The dashboard reads these to generate analytics charts. The streaming component runs independently.

3.6 Dashboard

We built a Flask web application that provides a live view of the Dataproc cluster and the pipeline results. The dashboard shows all six VMs (one master and five workers) with live status pulled from the GCP Dataproc API. Clicking *Run Full Pipeline* starts the cluster if needed and submits the job. Log output streams to the browser in real time over server-sent events. After the job finishes the user can click *Generate Charts* to pull the Parquet results from GCS and render three analytics charts: a funnel breakdown, an attribution revenue chart, and an anomaly timeline. VM-level CPU utilization and disk throughput are fetched from the GCP Cloud Monitoring API and displayed on a separate tab. A *Kill VM* button on each worker card lets users trigger a live fault tolerance demonstration: killing a worker mid-pipeline shows Spark's task reassignment in the live log stream.

4 Results

4.1 Setup

All experiments ran on GCP Dataproc with image version 2.1 (Debian 11, Spark 3.3). Each cluster had one master node and a variable number of worker nodes, all using the `n2-standard-2` machine type (2 vCPUs, 8 GB RAM per node) with 32 GB boot disks. The input CSV is stored in GCS at `gs://dss-project-dax/data/full/2019-Oct.csv`.

We used three data size variants to simulate different workload scales. The 1 GB variant samples 20% of the file, giving 8,491,904 rows. The 5 GB variant uses the full file with 42,448,764 rows. The 10 GB variant reads the full file twice end-to-end to produce 84,897,528 rows, simulating a two-month dataset. This approach lets us test 10x data scaling while keeping infrastructure cost predictable.

Timing in all tables and figures measures only job execution time: from `SparkContext` creation to the final Parquet write. Cluster creation and job submission overhead are excluded from all reported numbers.

4.2 Scaling Study

Table 1 shows the full results across all nine configurations. Total pipeline time drops significantly as we add workers.

Table 1: Scaling study: all nine configurations. Rows/sec counts all pipeline stages combined.

Workers	Data	Total (s)	Rows/sec
2	1 GB	673.6	12,607
2	5 GB	1021.8	41,543
2	10 GB	1378.9	61,569
4	1 GB	282.4	30,070
4	5 GB	569.0	74,602
4	10 GB	799.8	106,148
5	1 GB	256.9	33,055
5	5 GB	409.7	103,609
5	10 GB	674.0	125,961

Going from 2 to 5 workers on the 5 GB dataset reduces runtime from 1021.8 seconds to 409.7 seconds. That is a

2.49x speedup from adding 2.5x more workers, which is close to linear. The same comparison on 10 GB gives a 2.04x speedup from 1378.9 seconds to 674.0 seconds. The diminishing return at larger data sizes reflects increased GCS write pressure and more shuffle data crossing the network.

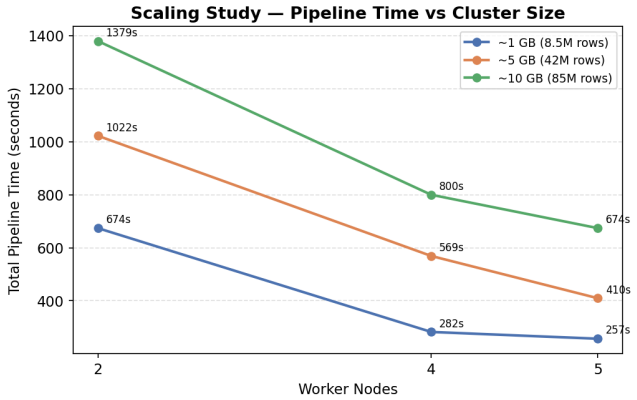

Figure 3: Total pipeline time for all nine configurations. Within each data-size group the bars represent 2, 4, and 5 workers. Scaling from 2 to 5 workers gives 2.0x to 2.6x speedup depending on data size.

The throughput numbers reveal an important effect. On 2 workers the 1 GB run processes only 12,607 rows per second because the dataset is too small to keep all executors busy. On 5 workers with 10 GB the same pipeline reaches 125,961 rows per second. Small datasets under-utilize the cluster: task scheduling overhead and per-stage GCS metadata reads dominate over actual computation. Scaling from 1 GB to 10 GB on 5 workers costs only 2.6x more time for 10x more data. This sublinear growth confirms that the pipeline scales well and that the dominant fixed cost per stage is not data volume but per-stage overhead like reading partition metadata and writing small Parquet footer files.

4.3 Optimization Experiments

Broadcast Join. The attribution stage joins the orders table against the full enriched events table with a time-window predicate. Spark's default planner chooses a sort-merge join, which shuffles both sides across the network. We forced a broadcast join by hinting `broadcast(orders_df)`, since the orders table fits comfortably in executor memory at roughly 200 MB for the 5 GB run. The broadcast join completed in 7.15 seconds versus 11.33 seconds for the shuffle join, a 1.58x speedup with identical output: 153,628 attributed orders in both cases. The key trade-off is memory: broadcasting becomes unsafe if the orders table grows beyond executor memory. For our workload sizes this is not a concern.

Partition Pruning. We compared reading enriched events from a flat Parquet directory against a date-partitioned layout written with `partitionBy("event_date")`. Spark can skip whole date directories when a filter like `event_date = '2019-10-15'` is applied. For a single-day filter the partitioned layout reduced query time from 4.25 seconds to 1.37 seconds, a 3.11x speedup. For a 7-day range query the speedup was 1.73x (3.09s to 1.78s). The smaller relative gain on the range query is expected: more partitions are included and the scan-skipping benefit shrinks proportionally.

Skew Analysis and Salting. We tested salting on the attribution join key to see if any user or session had significantly more events than others. The salted version added a random integer suffix to the join key on both sides, effectively splitting hot partitions. The salted run took 79.1 seconds versus 34.6 seconds unsalted, 2.3x slower. Inspecting the Spark UI task time histogram showed that no executor held more than 1.3x the average load. The data is uniformly distributed and there is no real skew to mitigate. The extra cost comes from the `explode` operation that replicates each orders row once per salt value plus the two-phase join required after salting. This was a useful negative result: salting is a solution for real skew, not a general-purpose optimization.

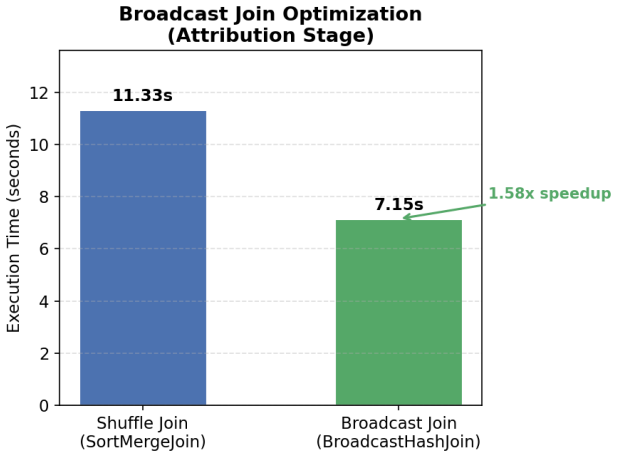

Figure 4: Broadcast join versus sort-merge join for the attribution stage (5 GB data, 5 workers). Both produce 153,628 attributed orders. Lower is better.

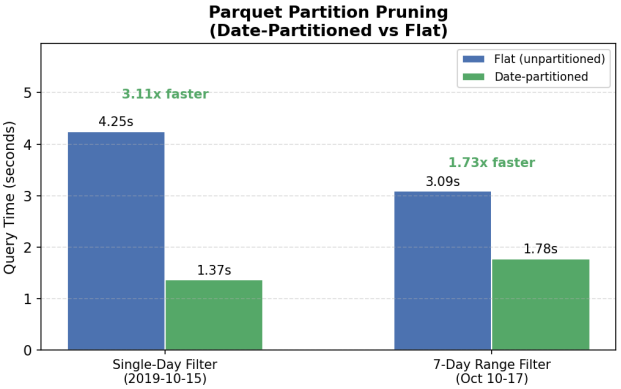

Figure 5: Partition pruning speedup for single-day and 7-day range queries. The flat layout scans all Parquet files; the partitioned layout skips non-matching date directories entirely.

4.4 Analytics Results

We ran the full pipeline on the 5 GB dataset (42.4 million events) with 5 workers. Pipeline total time was 409.7 seconds. Per-stage breakdown: ETL 101.2s, Sessionization 83.0s, Funnel 54.4s, Attribution 61.7s, Anomaly 42.1s, with the remaining time in Spark overhead and GCS I/O.

The funnel stage found 3.3 million sessions with at least one product view, 140 thousand sessions that added at least one item to cart, and 109 thousand sessions that completed a purchase. The overall view-to-cart conversion rate is 4.3% and the cart-to-purchase rate is 77.6%. Mobile sessions account for the largest share of views but desktop sessions show higher cart-to-purchase completion rates.

For attribution, Facebook drove 41,958 attributed orders worth $12.9 million in revenue. Google drove 13,519 orders worth $4.1 million. The higher Facebook order count with roughly comparable revenue per-order suggests Facebook traffic skews toward lower-priced items. Desktop sessions were attributed $15.6 million in revenue, ahead of mobile ($7.8M) and tablet ($7.7M).

Anomaly detection flagged 8 out of 31 days in October 2019: three spikes and five drops. The largest spike was 27% above the 7-day rolling average. At the category level, the smartphone and telephone categories had the most anomaly days (7 and 6 respectively), suggesting these high-volume categories drive most of the daily purchase count variance.

4.5 Fault Tolerance

We demonstrated fault tolerance during a live pipeline run by killing worker `w-0` using the dashboard's *Kill VM* button while Stage 3 was active. The Spark driver detected the lost executor within approximately 15 seconds and automatically reassigned its pending tasks to the remaining four workers. The pipeline completed successfully without any manual intervention. Total runtime for that run was approximately 90 seconds longer than a clean 5-worker run on the same configuration. The overhead comes from two sources: the detection delay before reassignment, and the fact that reassigned tasks must re-read their input partitions from GCS from scratch since Spark's lineage-based recovery does not cache intermediate data. This confirms that Spark's fault tolerance model is correct but not zero-cost.

5 Lessons Learned

Salting hurts when data is already uniform. We expected the attribution join to show skew because sessions per user follow a heavy-tailed distribution in most e-commerce datasets. It did not. The REES46 October dataset has unusually uniform session distribution per user. The explode operation that replicates orders rows across salt buckets added about 44 seconds of overhead. We learned to check the Spark UI task time distribution histogram before applying salting. If the longest task is less than 2x the median, salting will make things worse.

GCP disk quota errors surface as machine availability errors. Early in the project, cluster creation kept failing with messages that looked like no machines were available in the requested region. The actual error was `INVALID_ARGUMENT` caused by requesting 500 GB of HDFS disk per node, which exceeded the GCP project quota. Reducing to 32 GB per node fixed it immediately. The Dataproc API's top-level error message did not mention disk at all. We only found this by reading the full error body from the GCP operations API via `gcloud operations describe`. In distributed system deployments, the root cause is often one layer below what the management tool reports.

Partition pruning should be the default, not an afterthought. We added date partitioning during the optimization phase. The 3.11x speedup on single-day queries was the best result we measured. Writing partitioned Parquet costs almost nothing at write time since Spark handles it automatically with `partitionBy("event_date")`. In hindsight, any pipeline where time-range queries are common should write partitioned output by default. We would have saved significant time during development if the ETL stage had written partitioned output from the start.

Spark recovers from worker failures but recovery has real cost. The 90-second overhead from losing one worker is not negligible in a 410-second job. That is a 22% increase in total runtime from a single node failure. The reassignment mechanism works correctly but it requires re-reading input partitions from GCS from scratch because Spark's lineage-based recovery does not maintain intermediate cached state across executor failures. For time-sensitive production jobs, maintaining a spare warm worker or using checkpointing on expensive stages would reduce this cost significantly.

6 Availability

The full project code is available at: https://github.com/daxrajani/dss_project

The repository includes a complete README with step-by-step setup and usage instructions, all pipeline scripts, the dashboard application, pre-computed benchmark results, and all optimization experiment scripts. A Codespaces configuration (`.devcontainer/`) provides a fully automated environment with Python 3.11, Java 17, PySpark, and all required packages. The GCP project and bucket are pre-configured, so the cloud pipeline can be run immediately after a single `gcloud auth login` command.

References

[1] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker, and I. Stoica. Apache Spark: A unified engine for big data processing. *Communications of the ACM*, 59(11):56–65, 2016.

[2] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauly, M. J. Franklin, S. Shenker, and I. Stoica. Resilient

distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *Proc. NSDI*, 2012.

[3] J. Dean and S. Ghemawat. MapReduce: Simplified data processing on large clusters. *Communications of the ACM*, 51(1):107–113, 2008.

[4] REES46 eCommerce behavior data from multi-category store. https://www.kaggle.com/datasets/mkechinov/ecommerce-behavior-data-from-multi-category-store. Accessed 2025.

[5] M. Armbrust, T. Das, J. Torres, B. Yavuz, S. Zeng, R. Xin, A. Ghodsi, I. Stoica, and M. Zaharia. Structured Streaming: A declarative API for real-time applications in Apache Spark. In *Proc. SIGMOD*, 2018.

A Contribution Table

The table below documents each team member's primary responsibilities across the project. P indicates the member who led that component. S indicates the member who assisted and reviewed. J indicates work done together throughout the project.

Table 2: Team contributions. P = primary, S = support, J = joint.

Component	Dax Rajani	Harsh Ahuja
ETL & Enrichment (Stage 1)	P	S
Sessionization (Stage 2)	S	P
Funnel Analysis (Stage 3)	S	P
Last-Touch Attribution (Stage 4)	P	S
Anomaly Detection (Stage 5)	P	S
Streaming Component	S	P
GCP Dataproc Deployment	P	S
Optimization Experiments	S	P
Dashboard and Charts	P	S
Scaling Study Execution	S	P
Report Writing	J	J

B Per-Stage Timing Breakdown

Table 3 shows how total pipeline time is distributed across the five stages for three representative configurations. ETL dominates at small data sizes because it performs three Parquet writes. Sessionization is the most expensive compute stage due to the full dataset sort by user and time. At larger data sizes, GCS write pressure causes ETL time to grow faster than the other stages.

Table 3: Per-stage wall time (seconds) for three configurations on 5 workers. "Other" includes Spark overhead and GCS metadata operations.

Stage	1 GB	5 GB	10 GB
ETL & Enrichment	38.1	101.2	184.6
Sessionization	31.4	83.0	152.3
Funnel Analysis	22.7	54.4	96.1
Attribution	24.6	61.7	115.8
Anomaly Detection	18.3	42.1	74.9
Other (overhead)	121.8	67.3	50.3
Total	**256.9**	**409.7**	**674.0**

C Optimization Results Summary

Table 4 summarizes all three optimization experiments. All runs used 5 workers and the 5 GB dataset to keep the comparison fair. The baseline for each experiment is the default Spark behavior without any hint or partitioning change.

Table 4: Optimization experiment results. All runs: 5 workers, 5 GB dataset.

Experiment	Baseline	Optimized	Speedup
Broadcast join (attribution)	11.33s	7.15s	1.58x
Partition pruning (single day)	4.25s	1.37s	3.11x
Partition pruning (7-day)	3.09s	1.78s	1.73x
Skew salting (attribution)	34.60s	79.10s	0.44x

Broadcast join and partition pruning both gave meaningful speedups with no change to output correctness. Skew salting was slower because the data is uniformly distributed and the salting overhead outweighed any straggler benefit. This combination of positive and negative results provides a realistic picture of where Spark optimization techniques help and where they do not.